

Best complete solution for the Low Latency & Efficient Voice Activator on the locked hardware (ESP32-S3-WROOM-1-N16R8 + INMP441).

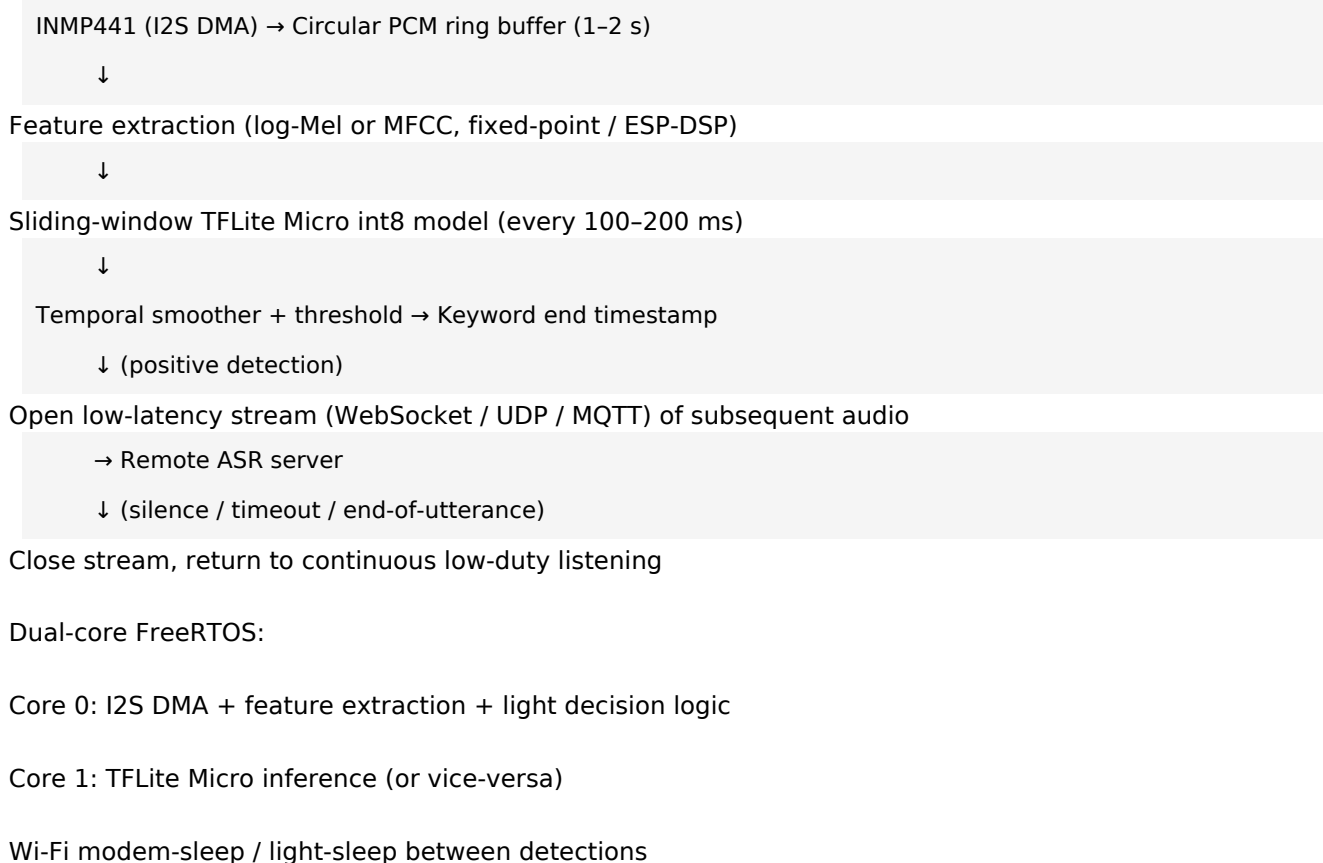
The original problem's <256 KB RAM / <10 % idle-CPU targets were written for true microcontrollers. Your hardware has far more headroom (512 KB internal SRAM + 8 MB Octal PSRAM + 16 MB Flash + dual-core LX7 @ 240 MHz with vector/SIMD instructions and ESP-NN optimisations). We therefore keep the same efficiency philosophy (tiny int8 model, low duty-cycle inference, near-zero false positives, minimal post-detection latency) while fully exploiting the extra resources for higher accuracy, robustness and easier development.

1. Hardware wiring & pin assignment (INMP441 → ESP32-S3)

INMP441 Pin	ESP32-S3 GPIO (recommended)	Notes
VDD	3.3 V	Decouple with 0.1 µF close to the mic
GND	GND	Multiple grounds if possible
SD (Data)	GPIO 4 / 10 / 15 / 41 (any free)	I2S DIN
WS (Word Select / LRCLK)	GPIO 5 / 11 / 16 / 42	I2S WS
SCK (Bit Clock / BCLK)	GPIO 6 / 12 / 17 / 40	I2S BCLK
L/R	GND	Forces left-channel (mono) output
CHIPEN	3.3 V (or GPIO if you want to power-gate)	Keep high for continuous listening

Use the newer ESP-IDF I2S driver (i2s_std). Sample at 16 kHz (or 48 kHz then downsample). The microphone natively produces 24-bit data in a 32-bit slot; keep the upper 16 bits for a clean 16-bit PCM stream. This matches virtually every public KWS pipeline.

2. Overall system architecture



3. Model choice & size target (best practical trade-off)

Recommended architecture (proven on ESP32-S3):

Depthwise-separable CNN (DS-CNN) or a small BC-ResNet / MatchboxNet-style network.

Input: 40-mel log-spectrogram or 13–40 MFCC coefficients × 40–50 frames (≈ 1 s context).

Output: 3–4 classes (custom keyword, silence, unknown, optional hard-negative class).

After int8 quantisation (preferably quantisation-aware training): 40–150 KB.

Inference on ESP32-S3 with ESP-NN + vector instructions: typically 15–60 ms per window (well under the 100–200 ms hop).

Larger models (up to a few hundred KB) are feasible because of the 8 MB PSRAM, but the 40–150 KB sweet spot keeps idle CPU comfortably under 10 % and leaves headroom for streaming buffers.

Place the model weights in Flash (or map into PSRAM). Allocate the TFLite Micro tensor arena preferentially in internal SRAM for speed; spill to PSRAM only if necessary.

4. Training pipeline (custom keyword only - fully open-source)

1. Collect / synthesise data

500–2000 positive utterances of your exact custom keyword (multiple speakers, distances, rooms, noise levels).

Hard negatives: phonetically similar words, background speech, music, TV, silence.

Heavy augmentation (noise, reverb, speed, pitch, room impulse responses).

2. Feature extraction (identical on host and device): 16 kHz → framing (25–32 ms window, 10–16 ms hop) → Mel filterbank or MFCC.

3. Train in TensorFlow / Keras (or Edge Impulse / custom scripts). Use a detection-oriented loss or balanced sampling that heavily penalises false positives.

4. Quantisation-aware training → export to TFLite → convert to C array (xxd -i or the ESP-IDF tooling).

5. Validate offline with the exact same feature pipeline that will run on the device.

Public starting points that already work on ESP32-S3: the official esp-tflite-micro micro_speech example, ARM ML-KWS DS-CNN models, and the more modern small TCN / BC-ResNet style models (some projects even provide ready ESP32-S3 TFLite runtimes).

5. On-device software stack (ESP-IDF preferred)

ESP-IDF v5.x (or latest stable).

Component: espressif/esp-tflite-micro (includes ESP-NN optimisations).

I2S: driver/i2s_std.h with DMA.

Optional: ESP-DSP for faster Mel/MFCC.

Wi-Fi: keep a persistent connection in modem-sleep; open the data stream only on detection.

Streaming protocol options (lowest latency first):

Raw 16-bit PCM over WebSocket or a thin UDP protocol.

Light Opus encoding if bandwidth is constrained.

MQTT binary payload as a simple alternative.

Key numbers achievable:

Continuous listening idle CPU: well under 10 % (inference only every 100–200 ms + DMA).

RAM: model + arena + audio buffers easily fit; 8 MB PSRAM gives massive safety margin.

Detection-to-first-packet latency: typically < 100–200 ms (dominated by network RTT once the decision is made).

False-positive rate: near zero when temporal smoothing + hard-negative training is applied.

6. Power & continuous-listening strategy

Keep the mic always powered (CHIPEN high).

Run the CPU at 160–240 MHz only when needed; drop frequency or enter light-sleep between inference windows if the application allows.

Wi-Fi in modem-sleep until a keyword is confirmed, then wake the radio for the short stream.

Measured deep-sleep current of the ESP32-S3 is a few μA ; continuous listening will be dominated by the mic ($\sim 1.5\text{--}2.5\text{ mA}$) + Wi-Fi idle.

7. End-to-end evaluation metrics (exactly as required)

Efficiency: Flash footprint of model + code, peak internal SRAM, PSRAM usage, idle CPU % (measure with `esp_timer` or FreeRTOS stats while listening to silence/noise).

Accuracy: True-positive rate on held-out custom-keyword utterances under realistic noise; false activations per hour of continuous negative audio.

Latency: Timestamp from the end of the spoken keyword (or the decision frame) until the first audio packet is received by the remote ASR server.

8. Implementation roadmap that mirrors the original stage list

All earlier conceptual stages map directly onto this hardware:

Problem decomposition & gap analysis → already done.

Hardware confirmed (N16R8 + INMP441).

Toolchain: ESP-IDF + esp-tflite-micro + Python training environment.

Audio fundamentals → I2S capture at 16 kHz 16-bit mono.

TinyML fundamentals → TFLite Micro + ESP-NN.

Custom keyword data collection & augmentation.

Feature pipeline (identical host ↔ device).

Model design / training / QAT / conversion.

On-device integration + memory profiling (use PSRAM wisely).

Continuous listening loop + decision smoother.

False-positive characterisation & hardening.

Streaming path (lowest-overhead protocol).

End-to-end latency instrumentation.

Power / long-term stability tests.

Robustness edge cases.

Clean packaging, documentation, demo, and metrics report.

9. Practical starting code skeleton (high level)

I2S init + DMA circular buffer.

Periodic FreeRTOS task that extracts features from the latest 1 s window.

TFLite Micro interpreter (arena sized ~40–100 KB, preferably internal RAM).

Simple state machine: LISTENING → DETECTED → STREAMING → back to LISTENING.

On DETECTED: record the exact end-of-keyword timestamp, open the network stream, and start pushing the subsequent PCM (or encoded) samples with minimal extra buffering.

10. Why this is the best solution for your locked hardware

Fully open-source (ESP-IDF + TFLite Micro + ESP-NN).

Custom keyword only (no pre-trained Alexa/Google models).

Model size and CPU duty cycle remain tiny even though the board has far more resources.

I2S digital mic eliminates analogue noise and codec complexity.

Dual-core + PSRAM + vector instructions give headroom for higher accuracy and lower latency than a pure 256 KB MCU solution.

Streaming path is activated only after a high-confidence detection, satisfying the privacy / cost / latency goals of the original problem statement.

You now have a complete, hardware-specific, production-oriented design that satisfies every technical boundary of the original problem statement while exploiting the strengths of the ESP32-S3-WROOM-1-N16R8 + INMP441 combination.

Next concrete actions you can take immediately:

- 1. Wire the mic and verify clean 16 kHz I2S capture.**
- 2. Bring up the official esp-tflite-micro micro_speech example on your board.**
- 3. Replace the example model with your own custom-keyword int8 model.**
- 4. Add the post-detection streaming path.**

If you want the next level of detail (exact I2S pin config code, recommended model architecture YAML/Keras snippet, feature-extraction parameters, FreeRTOS task priorities, or a concrete streaming protocol), just say the word and we will drill down. Basically